# 9.2 stringr: Regular Expressions

A pattern is a regular expression. Escape in R with two backslashes.

## 1. A pattern is a regular expression

Note 9.1 used literal text. A **regular expression** describes a set of strings. `stringr` (through `stringi`) uses ICU regex. Write the pattern as an ordinary R string, so backslashes double: the regex `\.` is `"\\."` in R.

`words` is 980 common English words that come with stringr. Use it for the examples.

```
library(tidyverse)

str_view(words, "ie")
```

`str_view()` shows the match. It is the fastest way to test a pattern.

---

## 2. The language, in pieces

`.` matches any single character (except a newline). Escape it when you want a real dot: `"\\."`.

```
str_view(c("abc", "a.c", "ac"), "a.c")
str_view(c("abc", "a.c", "ac"), "a\\.")
```

`^` is the start of the string. `$` is the end.

```
str_view(words, "^a")
str_view(words, "x$")
str_view(words, "^apple$")
```

A **character class** matches one character from a set: `[abc]`, `[a-z]`, `[0-9]`. `[^aeiou]` is "not a vowel." `[^ ]` is "not a space."

```
str_view(words, "^[aeiou]")
str_view(c("a b", "ab"), "[^ ]")
```

`(?i)` at the front makes the rest of the pattern case-insensitive.

```
str_detect(c("Apple", "BANANA", "pear"), "(?i)^a")
```

Repetition sits after the thing it repeats:

- `?` — 0 or 1
- `+` — 1 or more
- `*` — 0 or more
- `{n}` — exactly n
- `{n,}` — n or more
- `{n,m}` — between n and m

```
str_view(words, "^a.{3}$")
str_view(c("color", "colour"), "colou?r")
```

Parentheses make a **group**. `\\1` is "whatever group 1 matched." Note 4.2 used `matches("(.)\\1")` in `select()`. That is a repeated character.

```
str_view(words, "(.)\\1")
str_replace("apple", "(.)(.)", "\\2\\1")
```

---

## 3. Detect, count, extract, split, replace

All of these take the vector first, then the pattern.

```
x <- c("apple", "banana", "pear", "mango")

str_detect(x, "a")
x[str_detect(x, "^p")]
str_subset(x, "^p")

str_count(x, "a")
str_count("abababa", "aba")

str_extract(x, "[aeiou]")
str_extract_all(x, "[aeiou]")

str_split("a-b-c", "-")
str_replace(x, "[aeiou]", "-")
str_replace_all(x, "[aeiou]", "-")
```

`str_count()` counts **non-overlapping** matches. `"aba"` in `"abababa"` is 2, not 3.

`str_extract()` is the first match. `str_extract_all()` is every match, as a list.

`str_subset(x, pattern)` is `x[str_detect(x, pattern)]`.

---

## 4. `separate()` with a regex

`tidyr::separate()` splits one column into several. `sep` can be a regex, not only a fixed character.

```
tibble(x = c("a-1", "b.2", "c 3")) |>
  separate(x, into = c("letter", "num"), sep = "[^[:alnum:]]")
```

`[^[:alnum:]]` is any character that is not a letter or a digit: a hyphen, a dot, or a space.

---

## 5. Practice

Use `words`. Print the matches (or `str_view()` them).

1. Four-letter words that start with `a`.
2. Words that start with a vowel.
3. Words that end in `ed` but not `eed`.
4. Words that start with three consonants.